

总结：FAST 的三层贡献

算法层 *DCT + BPE*

DCT 把平滑动作曲线压缩约 10x, BPE 进一步学习动作"惯用语"
→ 350 个浮点数变成 30~60 个 token, 时序结构完整保留

工程层

在 LLM 看来这与输入其他单词没区别

把机器人控制完全统一到标准 LLM 训练管道

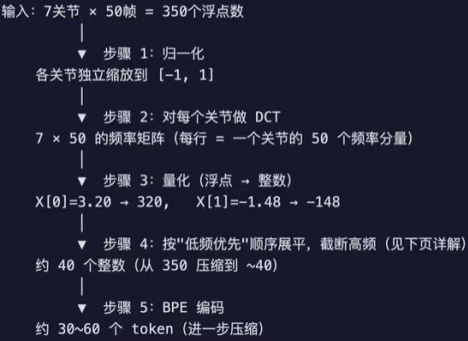
→ 训练速度 5x, LLM 的所有优化技巧直接可用

认知层

|"训练管道的小改动 (tokenization) 带来 5x 效率提升"

说明 VLA 训练还远未被充分优化——每个环节都可能藏着类似的提升空间

FAST 完整流程（5步）



初始输入：7关节 × 50帧 = 350个浮点数

- 含义：这代表一段典型的机器人手臂运动轨迹。7个关节（7 DoF）在未来 50 帧内的预期位置。
- 挑战：如果直接让大模型预测这 350 个浮点数，序列太长，计算极其缓慢，根本无法满足机器人需要的极低延迟（比如几十毫秒的反应时间）。

步骤 1：归一化 (Normalization)

- 操作：将各个关节的角度值独立缩放到 [-1, 1] 的标准区间。
- 目的：不同关节的物理旋转范围差异巨大（比如底座可能转 180 度，而手腕只转 30 度）。归一化消除了这种物理量纲的差异，让大模型在统一的数值尺度下学习，使训练更稳定。

步骤 2：对每个关节做 DCT（离散余弦变换）

- 操作：将时间维度的 50 帧轨迹，转换为频率维度的 50 个频率分量（形成 7 × 50 的频率矩阵）。
- 目的：这是 FAST 最核心的“魔法”。机器人的运动轨迹通常是平滑连续的。在信号处理中，平滑轨迹的能量主要集中在**低频部分**，而高频往往只是微小的抖动或噪音。DCT 成功将轨迹特征从“时间域”剥离到“频率域”。

步骤 3：量化 (Quantization: 浮点 → 整数)

- 操作：将 DCT 产生的浮点数乘以一个系数（如图例中的乘以 100），并四舍五入变成整数（如 3.20 → 320）。
- 目的：大语言模型的词表 (Vocabulary) 全都是离散的 ID（整数）。这一步将连续的物理信号彻底转化为离散的“词汇池”，为后续的李处理打下基础。

步骤 4：按“低频优先”顺序展平，截断高频

- 操作：按照频率从低到高将矩阵拉平成**一维数组，并直接砍掉高频部分**。
- 结果：从原本的 350 个数字，暴风骤雨般地压缩到了**约 40 个整数**。
- 目的：这就是为什么它叫“FAST”的原因。因为低频包含了动作的“主要轮廓”，扔掉高频（抖动/噪音）几乎不影响机器人的实际执行精度，却把模型的预测负担减轻了近 90%。这也极大降低了自回归预测 (Next-token prediction) 的推理延迟。

步骤 5：BPE 编码 (Byte Pair Encoding)

- 操作：使用自然语言处理 (NLP) 中最经典的 BPE 算法。它会寻找经常一起出现的“整数对”，并把它们合并成一个新的、更高级的 Token。
- 目的：进一步压缩序列长度，最终将一段复杂的机器人 50 帧动作，浓缩成了**仅仅 30~60 个 Token**。此时，这些动作彻底变成了类似于英语单词一样的“动作词汇”。



步骤 5：BPE 是什么？

BPE (Byte Pair Encoding) — 与 GPT 分词器完全相同的算法，核心思想：

反复找出序列中最高频的相邻数字对，将其合并为一个新符号

三步理解 BPE：

- 第一步：初始状态，每个整数都是独立的 token
[..., 320, 185, 94, -148, -87, 320, 185, 94, -148, ...]
- 第二步：统计所有相邻对的频率，找最高频的合并
"320, 185" 出现最多 (8万次) → 新增符号 #A, 替换所有出现
[..., #A, 94, -148, -87, #A, 94, -148, ...]
- 第三步：重复——再找最高频对
"#A, 94" 出现最多 (5万次) → 新增符号 #B
[..., #B, -148, -87, #B, -148, ...]

重复数次后，词表里的每个 token 都对应一段常见的动作频率模式，
序列从 ~40 个整数进一步压缩到 **30~60 个 token**

局限性：推理速度变慢

自回归解码无法并行

Flow Matching (π_0)：一次性并行生成所有值

输入噪声 → [迭代10步] → 350维动作 （约 10ms）

FAST (π_0 -FAST)：必须逐 token 串行生成

→ token₁ → token₂ → ... → token_{s0} （约 30~50ms）

为什么不能并行？

第 i 个 token 的预测**依赖前 $i - 1$ 个 token**——这是自回归模型的本质。

团队的判断：语言模型领域已有成熟的加速方案（KV Cache、Speculative Decoding），这是一个已知的、可在工程上解决的问题

两种方法的完整对比

维度	简单分箱	Flow Matching	FAST（本文）
序列长度	350 token	不适用	30~60 token
时序连贯性	无	有	有（DCT 保证）
精细任务	失败	成功	成功
训练速度	慢	慢	快 5x
推理速度	慢	快	慢
LLM 生态兼容	部分	否	完全兼容

后续的 Knowledge Insulation（EP06）把 FAST 和 Flow Matching 融合，尝试同时获得两者的优点